DOMAIN-DRIVEN IoT COLD CHAIN LOGISTICS

Technical Report

ColdChain IoT Dispatch System

C++17 / REACT 19 / ESP32 SIMULATION / JSON STORAGE

Software engineering and application development

Monitoring, temperature alerts and recorded dispatch responses

Revision grounded in the supplied repository

This edition updates the supplied report against the C++ implementation in **Nezine/cold-chain-esp32-logistics**, commit **0a65a45**. It distinguishes implemented behavior, automated checks and work that still requires validation.

42	21	3
Tests found in source	Tests rerun successfully here	Jobs defined in backend.yml

The 21 rerun tests comprise 18 frontend tests and 3 simulator tests. The other 21 tests exercise the C++ backend, API, bridge and legacy dispatch code; they were inspected but not rerun in this Windows environment.

Document control

Revision date: 30 October 2026
Scope: C++ reference implementation and its React client
Language: English, following the supplied report
Source: github.com/Nezine/cold-chain-esp32-logistics

The separate local Python/SQLite demo, its screenshots and its 26-test result are not evidence for this C++ version.

Scope and revision summary

The assignment asks for a platform that monitors and dispatches temperature-sensitive goods, demonstrates Domain-Driven Design principles, uses Observer for temperature alerts and Repository for persistence, and can simulate sensor data without physical hardware.

The implemented demonstration tracks shipments, accepts temperature readings, classifies severity, retains alerts and records what a dispatcher did. Here, **dispatcher** means the operator who reviews an alert and records an operational response, such as moving goods to a backup cooler.

Implemented in this C++ revision	Outside the implemented scope
Shipment creation through HTTP API / CLI	An active shipment-creation form in the current UI
Reading validation, severity and alert history	Automatic sensor timeout or guaranteed delivery
One recorded response per alert	Vehicle assignment, route optimization or delivery lifecycle
Local JSON persistence and process locking	Database server, authentication or production deployment

Corrections to the supplied report

The active React App does not render ShipmentForm and does not use useLogistics. Those files remain in the tree, but the active service does not export their createShipment / recordResponse dependencies. Shipment creation must be demonstrated through the API or CLI for this revision.

The previous blanket claim of 42 passing tests is replaced by a source inventory and explicit execution results. Unverified timings, firmware memory percentages, team-size and code-volume metrics are omitted. Workflow configuration is not presented as proof of a successful CI run.

Reading guide

Pages	Contents
3-4	Runtime architecture; domain model and OOP patterns
5-6	HTTP contract; persistence and consistency
7-8	Active frontend; sensor simulation and bridge
9-10	Verification evidence; setup and repeatable demo
11-12	Acceptance checklist and limitations; source traceability

Runtime architecture

C++ owns the classification, alert creation, validation and stored state. A small Python HTTP adapter translates requests into CLI calls. The React application reads the API and records responses; it does not implement the authoritative temperature policy.

Layer	Responsibility / concrete implementation
React client	App.js polls shipments and alerts approximately every 2 seconds. The CRA development proxy forwards /api to 127.0.0.1:8080.
HTTP adapter	tools/dev_api.py uses ThreadingHTTPServer. Each supported request invokes the C++ executable with an argument array and a 15-second subprocess timeout.
Application and domain	LogisticsService coordinates validation, Shipment classification, ReadingPublisher and AlertMonitor. backend_cli.cpp exposes the operations.
Repository	StateRepository abstracts load/save. FileStateRepository locks the local store and writes a version-1 JSON snapshot.
Sensor input	sensor_bridge.py translates firmware-format lines into POST /api/readings. It accepts stdin fixtures or an optional USB serial port.

End-to-end flow

```
ESP32 / JSONL fixture
  -> Python sensor bridge
  -> HTTP POST /api/readings
  -> Python adapter -> C++ CLI -> LogisticsService
  -> validation -> classification -> Observer -> JSON snapshot
  -> React GET /shipments + GET /alerts -> dashboard
```

A response follows a separate command: the user selects an alert, chooses an action and submits a note. The backend records the response and changes responseStatus to responded. The measured temperature and original reading remain unchanged.

Deployment boundary

The adapter binds to 127.0.0.1 by default. Native Windows is not the documented C++ runtime: the repository uses POSIX flock, file descriptors and rename. Use Linux or a configured WSL Linux environment; macOS portability still requires verification.

Domain model and OOP patterns

Concept	Role and invariant
Shipment	Identified by a generated positive integer. Stores name, min, max and criticalMargin. A new shipment has no temperature and connection=loading.
Reading	Carries shipmentId, deviceId, globally unique readingId, finite temperature, unit C and a canonical UTC time.
Alert	One record for each accepted non-normal reading. Links to the shipment and reading; starts pending.
Response	Targets one alert belonging to one shipment. Holds action, optional note and server recordedAt. At most one response per alert.

DDD principles: vocabulary and behavior boundaries

Shipment, reading, alert and response form a common domain vocabulary. Shipment::classify encapsulates the temperature decision; LogisticsService coordinates use cases through a repository abstraction. This is a small domain-oriented design, not a claim of a complete strategic DDD implementation. Much of the stored model still uses JSON objects rather than strongly typed entities.

Observer: notify when a reading is accepted

ReadingPublisher stores references to ReadingObserver implementations. AlertMonitor implements onReading and appends an alert only when severity is warning or critical. Delivery is synchronous within the C++ command. It is not a message broker or a WebSocket channel.

Repository: separate use cases from storage

StateRepository defines virtual load and save methods. FileStateRepository supplies the JSON implementation. LogisticsService receives the interface by reference, so storage responsibilities are separated from use-case orchestration.

C++ resource management

The repository is non-copyable and releases its process lock in its destructor. JSON ownership uses smart-pointer helpers with the JSON-C release function. Virtual destructors support safe polymorphic cleanup. These RAII mechanisms reduce manual cleanup paths.

Exact code identifiers are used here. The frontend architecture card displays illustrative labels and a hard-coded observer count of 4; those labels are not an inventory of backend classes or registered observers.

HTTP contract and temperature policy

All routes use the /api prefix. POST bodies are UTF-8 JSON with Content-Type: application/json and a maximum size of 64 KiB. Error bodies use {"error":"..."}.

Method and route	Success / behavior
GET /health	200; status, backend=cpp and shipment count
GET /shipments	200; all shipment summaries
POST /shipments	201; create and return a shipment
GET /shipments/:id/readings	200; full reading history for that shipment
POST /readings	201; return the updated shipment
GET /alerts	200; all pending and responded alerts
POST /shipments/:id/responses	200; record response to the specified alertId

Chosen classroom policy: min=2, max=8, margin=2

Temperature t (degrees Celsius)	Severity	Examples
2 <= t <= 8	normal	2, 5, 8
0 <= t < 2 or 8 < t <= 10	warning	0, 1, 9, 10
t < 0 or t > 10	critical	-1, 12

For custom ranges, critical means outside [min - criticalMargin, max + criticalMargin]; warning means outside [min,max] but inside that extended interval. These are demo thresholds, not a validated storage specification for every pharmaceutical or food product.

Request examples

```
POST /api/shipments
{"name":"Demo shipment","min":2,"max":8,"criticalMargin":2}

POST /api/shipments/1/responses
{"alertId":2,"action":"moved_to_backup_cooler",
 "note":"Moved to a backup cooler"}
```

Validation and persistent state

Input / operation	Rule implemented by the C++ backend
Shipment configuration	Nonblank name: 1-256 UTF-8 bytes. -100 <= min < max <= 150. 0 < criticalMargin <= 100; default margin 2.
Reading	Required positive shipmentId; nonblank deviceId and readingId (up to 256 bytes). Temperature is a finite number in [-100,150]; unit is C.
Time and duplicates	Exact YYYY-MM-DDTHH:MM:SSZ with valid calendar fields. Older times for the same shipment return 409; equal times with different IDs are accepted. Repeated readingId returns 409.
Response	Action is nonblank, up to 256 bytes. Optional note up to 4096 bytes. Wrong shipment/alert returns 404; second response returns 409.

Snapshot transaction

The JSON snapshot has version=1 and four arrays: shipments, readings, alerts and responses. A CLI process acquires an exclusive lock on the adjacent .lock file before loading state. It validates and mutates in memory, writes a temporary snapshot, calls fsync, then renames the temporary file over the target. The lock is released when the repository is destroyed.

A missing snapshot starts an empty store. Invalid JSON, unsupported versions, missing arrays or non-object array entries are rejected. These structural checks are not a complete referential-integrity audit of every record. Corrupt files are not silently replaced with an empty store.

What survives and what is retained

Shipment history contains at most the latest six temperatures for display. The reading collection retains all accepted readings; alerts and responses remain available after process restart. Returning to a normal temperature does not remove old alerts or automatically mark them responded.

The design provides local process coordination and atomic snapshot replacement. It does not establish database-grade durability under every power-loss or network-filesystem scenario; the containing directory is not fsynced after rename.

Error status summary

400 invalid input; **404** missing route or record; **409** duplicate/conflict; **413** oversized body; **415** unsupported content type; **500** storage/state error; **503** unavailable or timed-out C++ subprocess.

Frontend: actual integrated behavior

The active React client defaults to the real API. Set REACT_APP_USE_MOCK=true only for the optional in-memory demonstration. The live API path does not run the local temperature simulation.

Screen / mechanism	Current behavior
Dashboard	Shipment summaries and selected-shipment detail; severity comes from the backend. A new shipment displays no data until its first reading.
Shipments	Displays the list of shipment cards. It does not currently render an active creation form.
Alerts and response	Shows historical alerts and response state. The detail panel sends alertId, action and optional note for the selected shipment.
Reports	Computes the mean of latest numeric temperatures where connection=ok. If no shipments qualify, the current calculation displays 0.0 degrees C.
Polling and outages	App.js polls every 2 seconds, guards against overlapping refreshes and queues a refresh requested during an active one. It keeps last data with an outage message and retries.
Live feed	The frontend subject announces previously unseen alert IDs. The feed retains up to 8 entries; server alert history remains separate.

Creation: API support is not a visible form

ShipmentForm.jsx and useLogistics.js exist but are not imported by the active App. The hook references createShipment and recordResponse, while the active shipmentService exports respondToAlert and read operations. Their presence in the repository therefore does not establish an available create-shipment user flow.

For the C++ presentation, create records using POST /api/shipments, then show them arriving on the dashboard. The earlier screenshots showing a creation form belong to the separate Python/SQLite demo and should not be labeled as this C++ revision.

Interpretation of status

Responded means that the operator recorded an action; it does not mean temperature recovered. Sensor silence is also different from HTTP failure: the backend does not currently change connection=ok to timeout when readings stop.

ESP32 simulation and bridge

Hardware is optional for the core demonstration. A JSONL fixture can drive the same bridge used for serial input, allowing the HTTP adapter and C++ domain workflow to be exercised without a physical probe.

Two simulation paths

Source	Purpose / output
ESP32 firmware	Arduino code emits JSON over USB serial at 115200 baud, every 2 seconds. The fixed cycle is 2.0, 8.0, 8.1, 10.0 and 10.1 degrees C.
Firmware-format fixture	02-domain-backend/examples/firmware-demo.jsonl supplies 5, 9 and 12 degrees C for the repeatable end-to-end demonstration.
Legacy Python simulator	04-esp32-sensor/examples/simulate_sensor.py has its own deterministic format and 3 tests. Its output must not be assumed to match the HTTP reading contract directly.

Translation into the API contract

The bridge maps a firmware label to an explicitly configured numeric shipment ID. It checks the configured device and nonnegative sequence/uptime fields, then constructs readingId as device:session:sequence. It assigns host UTC receive time; board uptime is not treated as wall-clock time.

```
{"shipmentId":1,"deviceId":"device1",
 "readingId":"device1:demo-boot:1","temperature":9,
 "unit":"C","time":"2026-10-30T10:00:01Z"}
```

Boot messages that are not JSON are skipped. Oversized lines above 4096 bytes are rejected. The C++ API still validates the temperature, unit, ID uniqueness and time ordering. Backend rejections are reported by the bridge rather than silently accepted.

Session and hardware limitations

A new session ID is generated by default when the bridge starts. After a board reboot, restart the bridge with a new session so reused sequence numbers do not collide. A deliberately stable session is useful for replay/duplicate testing. There is no durable retry queue or exactly-once guarantee across reboot and network failure.

Optional USB input requires pyserial and a physical board. Physical probe accuracy, Wi-Fi transport, GPS tracking, OTA updates and real cold-chain deployment have not been validated in this revision review.

Verification evidence

Source under review: commit 0a65a451af7f43424c4d11d9619e8fe9d68ee420. The revision review was performed on Windows on 30 October 2026. Test counts below distinguish code inventory from executed results.

Check	Count	Evidence in this review
C++ core tests	11	Source inspected; not executed here
HTTP adapter / bridge tests	2	Source inspected; not executed here
Legacy dispatch tests	8	Source inspected; not executed here
React tests (4 suites)	18	PASS: 18/18
Python simulator tests	3	PASS: 3/3
React production build	-	Compiled successfully
C++ restart demo / browser smoke	-	Scripts inspected; not rerun here
ESP32 firmware build / hardware	-	Not rerun / not tested here

Total inventory: 42 tests. Executed here: 21 tests, all passed. C++ tests require a POSIX environment and the JSON-C toolchain. WSL was checked, but no installed Linux distribution was listed. No backend pass result is inferred from the local Python/SQLite demo.

Frontend execution details

The C++ reference source was extracted to a separate directory. Tests and production build used the existing installed frontend node_modules via a directory junction. This validates that source with the available dependencies; it is not a clean npm ci reproducibility check against the reference lockfile.

CI configuration and coverage limits

backend.yml defines backend, firmware and frontend jobs on Ubuntu. It runs setup/check/demo, a pinned PlatformIO firmware build, and frontend tests/build respectively. The separate sensor workflow also tests simulator examples. A workflow definition alone does not prove a completed successful run; no CI success is asserted here.

Source tests cover boundaries, persistence, rejected mutations, concurrent creates/responses, invalid payloads and transport errors. Load capacity, every browser, accessibility, real sensor hardware and production security remain outside this evidence.

Run and demonstrate the C++ version

Use a separate checkout of the referenced C++ commit on Linux or configured WSL. These commands do not describe the earlier local Python/SQLite server on ports 8000/8001. Run from the repository root unless specified.

1. Prepare and verify

```
bash 02-domain-backend/tools/setup.sh
bash 02-domain-backend/tools/check.sh
python3 02-domain-backend/tools/demo.py
```

Prerequisites: Python 3.10+, Git, g++ with C++17, make, pkg-config and network access for setup. Setup pins CMake 3.31.6, PlatformIO 6.1.19 and JSON-C 0.19. demo.py uses a temporary store and its own server, then restarts the API to assert persistence.

2. Start the interactive backend and frontend

```
# Terminal A
bash 02-domain-backend/tools/start.sh

# Terminal B
cd 03-frontend/exercises/esp32
npm ci
BROWSER=none HOST=127.0.0.1 npm start
```

Open http://127.0.0.1:3000. The adapter listens on 8080. Default state is 02-domain-backend/build/data/state.json. start.sh accepts --port and --state; a custom state parent directory must exist.

3. Create a shipment through the API

```
curl -sS http://127.0.0.1:8080/api/shipments \
  -H 'Content-Type: application/json' \
  -d '{"name":"Demo A","min":2,"max":8}'
```

Read the returned id; do not assume it is 1 when the store already contains shipments. Repeat with another name to show that multiple shipments are supported, then select the intended shipment in the dashboard.

4. Send the fixture and record a response

```
python3 02-domain-backend/tools/sensor_bridge.py \
  --shipment-id <returned-id> --session-id demo-boot \
  < 02-domain-backend/examples/firmware-demo.jsonl
```

The final temperature is 12 degrees C, with warning and critical alerts. Select the critical alert, record an action, and show that the measured value remains 12. Restart Terminal A and reload the UI: the response and reading should remain. Use a fresh session ID on a new demo run.

Acceptance checks and next steps

Scenario	Expected result to show
Create A and B through the API	Two distinct IDs; both appear on the dashboard after polling. Names alone are not record identity.
A has no readings yet	temperature=null and connection=loading; no invented initial temperature.
Send A: 5 -> 9 -> 12 degrees C	normal -> warning -> critical; two historical alerts. B remains unchanged.
Respond to A's critical alert	responseStatus=responded; 12 degrees C and the original reading are retained.
Replay a reading ID	409 conflict; no extra reading or alert is created.
Send a new normal reading	Current severity returns to normal; previous alerts remain in history.
Restart adapter with same state	Shipments, readings and recorded responses are still present.
Stop adapter temporarily	UI keeps last data, displays an outage message and retries.

Presentation split for one or two speakers

Speaker 1: introduce the problem and dispatcher role; explain the C++ domain service, Observer and Repository with the 5/9/12 example. **Speaker 2:** operate the API/UI demo, show the recorded response and restart behavior, then state the validation limits. One speaker can follow the same order. The table above is a rehearsal checklist, not a claim that these scenarios were rerun in this review.

Prioritized improvements

First: wire a supported creation form into the active client, add consecutive-create and selection tests, and replace illustrative architecture-card labels with accurate names. **Next:** add explicit sensor-staleness policy and clearer empty-report output. **For wider use:** add authentication, operational audit identity, pagination, durable storage/backup and reliable sensor delivery.

Vehicle scheduling and route optimization would need additional domain requirements. The present dispatch feature records an operator response to an alert; it does not implement transport planning.

Source traceability and revision record

Canonical repository:
https://github.com/Nezine/cold-chain-esp32-logistics
Reviewed commit:
0a65a451af7f43424c4d11d9619e8fe9d68ee420

Paths below are relative to the repository at that commit. The source code takes precedence when a handoff note or README describes an earlier integration state.

Report topic	Primary source files
Domain, Observer, Repository	02-domain-backend/src/backend.h 02-domain-backend/src/backend_cli.cpp 02-domain-backend/src/json_support.h
HTTP transport and contract	02-domain-backend/tools/dev_api.py 02-domain-backend/docs/API_V1.md
Setup and end-to-end rehearsal	02-domain-backend/tools/setup.sh 02-domain-backend/tools/setup_json_c.sh 02-domain-backend/tools/check.sh 02-domain-backend/tools/demo.py
Frontend integration and creation gap	03-frontend/exercises/esp32/src/App.js src/pages/Shipments.jsx src/services/shipmentService.js src/components/ShipmentForm.jsx src/hooks/useLogistics.js
Sensor and optional browser test	04-esp32-sensor/src/main.cpp 02-domain-backend/tools/sensor_bridge.py 03-frontend/exercises/esp32/tools/browser_smoke.py
Tests and CI	02-domain-backend/tests/test_*.py 04-esp32-sensor/tests/test_simulate_sensor.py 03-frontend/exercises/esp32/src/**/*.test.js .github/workflows/backend.yml

Revision record: 30 October 2026

Retained the requested C++17 / JSON architecture. Corrected the create-shipment UI claim, distinguished live code from unused components, separated tested results from test inventory, removed unsupported metrics, and clarified platform, persistence, sensor and dispatch limitations. Added a repeatable C++ demo checklist for one or two presenters.

This is a source-grounded academic technical report, not a certification of production readiness or a record of lecturer acceptance. The original supplied PDF is preserved unchanged.